EGCI427 · WEB PROGRAMMING

WEEK 04

RESTful API Design with MongoDB & Mongoose

Connecting Express to a real database with a full CRUD API

2-HOUR LECTURE

Node.js · Express · MongoDB · Vue.js 3

Today’s Learning Objectives

1. Connect an Express app to MongoDB (Atlas or local)
2. Define Mongoose schemas and models
3. Build a full CRUD REST API backed by MongoDB
4. Add query filters, sorting, and validation

AGENDA

BEGINNER

MongoDB Setup & Connection

0:00 – 0:40

INTERMEDIATE

Mongoose Schema & Create/Read Routes

0:40 – 1:20

ADVANCED

Update, Delete & Query Filters

1:20 – 2:00

PART A

BEGINNER

MongoDB Setup & Connection

Create a free MongoDB Atlas cluster and connect Mongoose to it.

0:00 – 0:40

BEGINNER

MongoDB Atlas Setup (Cloud, Recommended)

1

Sign up free

mongodb.com/cloud/atlas/register — create an M0 (free) cluster.

2

Database Access

Create a database user with a password.

3

Network Access

Add 0.0.0.0/0 to allow connections from anywhere (class use).

4

Copy connection string

Connect → Drivers → copy the mongodb+srv:// URI.

BEGINNER

Install & Configure the Project

terminal

```
mkdir egci427-week4 && cd egci427-week4
npm init -y
npm install express mongoose dotenv
npm install --save-dev nodemon

echo "PORT=3000
MONGO_URI=mongodb+srv://<user>:<pass>@cluster0.xxxxx.mongodb.net/egci427" > .env
```

BEGINNER

Connect to MongoDB in server.js

server.js

```
import mongoose from 'mongoose';
import dotenv from 'dotenv';
dotenv.config();

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('✅ MongoDB connected'))
  .catch(err => console.error('❌ MongoDB error:', err.message));
```

💡 Run npm run dev and confirm you see "✅ MongoDB connected" in the terminal.

PART B

INTERMEDIATE

Mongoose Schema & Create/Read Routes

B

Model the Product resource and implement Create + Read.

0:40 – 1:20

INTERMEDIATE

models/Product.js

models/Product.js

```
import mongoose from 'mongoose';

const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  price: { type: Number, required: true, min: 0 },
  category: { type: String, default: 'general' },
  inStock: { type: Boolean, default: true },
}, { timestamps: true });

export default mongoose.model('Product', productSchema);
```

INTERMEDIATE

routes/products.js — Create & Read

routes/products.js

```
router.post('/', async (req, res) => {
  try {
    const product = await Product.create(req.body);
    res.status(201).json(product);
  } catch (err) { res.status(400).json({ error: err.message }); }
});

router.get('/', async (req, res) => {
  const products = await Product.find();
  res.json(products);
});

router.get('/:id', async (req, res) => {
  const product = await Product.findById(req.params.id);
  if (!product) return res.status(404).json({ error: 'Not found' });
  res.json(product);
});
```

INTERMEDIATE

Test with curl

terminal

```
curl -X POST http://localhost:3000/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop","price":25000,"category":"electronics"}'

curl http://localhost:3000/products
```

PART C

ADVANCED

Update, Delete & Query Filters

Complete the CRUD cycle and support filtering/sorting via query strings.

1:20 – 2:00

ADVANCED

Update & Delete Routes

routes/products.js

```
router.put('/:id', async (req, res) => {
  const product = await Product.findByIdAndUpdate(req.params.id, req.body,
    { new: true, runValidators: true });
  if (!product) return res.status(404).json({ error: 'Not found' });
  res.json(product);
});

router.delete('/:id', async (req, res) => {
  const product = await Product.findByIdAndDelete(req.params.id);
  if (!product) return res.status(404).json({ error: 'Not found' });
  res.json({ message: 'Deleted', product });
});
```

ADVANCED

Filtering, Range Queries & Sorting

routes/products.js

```
router.get('/', async (req, res) => {
  const { category, minPrice, maxPrice, sort } = req.query;
  const filter = {};
  if (category) filter.category = category;
  if (minPrice || maxPrice) {
    filter.price = {};
    if (minPrice) filter.price.$gte = Number(minPrice);
    if (maxPrice) filter.price.$lte = Number(maxPrice);
  }
  let query = Product.find(filter);
  if (sort) query = query.sort(sort); // e.g. ?sort=-price
  res.json(await query);
});
// Try: /products?category=electronics&minPrice=1000&sort=-price
```

ADVANCED

Challenge (Take-Home)

- Add a reviews sub-document array to the Product schema
- Each review needs:
  - user, rating, comment
- Create POST /products/:id/reviews to add a review

Submission Checklist

Confirm each item before you leave the lab / submit your work

- [ ] MongoDB Atlas cluster created and connection string in .env
- [ ] Server logs "✅ MongoDB connected" on startup
- [ ] models/Product.js defines schema with validation
- [ ] All 5 CRUD operations tested and working
- [ ] Query filters (category, price range, sort) work correctly
- [ ] MongoDB Compass/Atlas shows saved documents

RECAP

What We Covered This Week

- Connecting Express to MongoDB with Mongoose
- Defining schemas with validation rules
- Implementing full CRUD against a real database
- Query-string filtering, range queries, and sorting

📂 **Hands-on Lab**

Week04_Lab_MongoDB_Mongoose_CRUD.docx — complete Parts A–C hands-on

NEXT WEEK: Authentication, Validation & Error Handling — JWT and bcrypt